REGISTRO DE EVIDENCIAS

Almacenamiento distribuido de archivos con gRPC

Asignatura: Sistemas Distribuidos

Presentado por:

Senén Dario Niño Martinez

Santiago Niño Villalobos

Edgar Andres Alvarez

Andres David Torres

Presentado a:

MAGDA LORENA PINEDA RODRIGUEZ

Fundación Universitaria Juan de Castellanos

Tunja, Boyacá – noviembre de 2026

Introducción

Este documento reúne las evidencias de las pruebas realizadas al sistema de almacenamiento distribuido. Cada evidencia indica qué se hizo, qué se observa en la captura y qué demuestra sobre el funcionamiento del sistema.

Las pruebas se ejecutaron entre el 20 de octubre y el 12 de noviembre de 2026, en equipos con Windows y Python 3.14.8. El coordinador escucha en el puerto 50050, los nodos en los puertos 50051, 50052 y 50053, y el gateway web en el 8000. La comunicación entre coordinador, nodos, cliente y gateway es siempre por gRPC.

Resumen de evidencias

ID	Evidencia	Resultado
E1	Registro de nodos en el coordinador	Correcto
E2	Chequeo de salud periódico	Correcto
E3	Comunicación directa con un nodo	Correcto
E4	Subida con réplicas, listado y descarga	Correcto, hash OK
E5	Consulta de un archivo inexistente	Error controlado `NOT_FOUND`
E6	Caída de un nodo durante la descarga	Descarga desde la réplica
E7	Detección de la caída por el coordinador	Correcto
E8	Recuperación de los nodos	Correcto
E9	Panel web: nodos, subida, información y descarga	Correcto, hash OK
E10	Panel web: nodo caído	Tarjeta e insignias en rojo
E11	Panel web: historial de caída y recuperación	Correcto
E12	API: consulta exitosa	HTTP 200
E13	API: archivo inexistente	HTTP 404
E14	Mejora: confirmación de subida	Solo se registran réplicas exitosas
E15	Mejora: eliminación de archivos	Borrado en todas las réplicas disponibles
E16	Acceso remoto al panel por Tailscale	HTTP 200 desde el celular
E17	Panel en una red lenta (4G y relevo)	Fallos de red; motivó una mejora
E18	Nodo en un segundo computador	Registro y chequeo de salud entre equipos
E19	Re-replicación automática	Copia verificada tras 31 s de gracia

ID	Evidencia	Resultado
E20	Estado después de la re-replicación	Tres réplicas activas
E21	Resincronización al volver un nodo	Copias sobrantes eliminadas
E22	Indicador de réplicas en riesgo	Archivos marcados "en riesgo"
E23	Acceso móvil con un usuario sin permisos de eliminación	Sin botón Eliminar

1. PRUEBAS DESDE LA TERMINAL

Estas pruebas usan el cliente de consola (cliente.py) y los registros de cada componente. Cada componente se ejecutó en su propia terminal de Visual Studio Code.

E1. Registro de nodos en el coordinador

Qué se hizo: Se inició el coordinador y después nodo1 y nodo2.

Qué se observa: A las 14:24:29 el coordinador queda escuchando en el puerto 50050. A las 14:24:41 y 14:24:51 registra a nodo1 y nodo2 con sus direcciones. Cada nodo se anuncia al arrancar mediante la llamada gRPC `RegistrarNodo`.

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python coordinador.py
2026-10-20 14:24:29,291 [coordinador] Coordinador escuchando en el puerto 50050
2026-10-20 14:24:41,110 [coordinador] Nodo registrado: nodo1 en localhost:50051
2026-10-20 14:24:51,278 [coordinador] Nodo registrado: nodo2 en localhost:50052
```

Figura 1. Registro de nodo1 y nodo2 en el coordinador

E2. Chequeo de salud periódico

Qué se hizo: Con el coordinador y los nodos activos, se observaron los registros de cada nodo.

Qué se observa: Cada nodo recibe un `Ping` del coordinador cada 5 segundos: nodo1 entre las 14:25:24 y las 14:26:04, y nodo2 entre las 14:26:24 y las 14:27:04. Así el coordinador sabe en todo momento qué nodos siguen vivos.

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python nodo.py --id nodo1 --puerto 50051
2026-10-20 14:25:24,649 [nodo1] Ping recibido desde coordinador
2026-10-20 14:25:29,662 [nodo1] Ping recibido desde coordinador
2026-10-20 14:25:34,672 [nodo1] Ping recibido desde coordinador
2026-10-20 14:25:39,684 [nodo1] Ping recibido desde coordinador
2026-10-20 14:25:44,697 [nodo1] Ping recibido desde coordinador
2026-10-20 14:25:49,711 [nodo1] Ping recibido desde coordinador
2026-10-20 14:25:54,725 [nodo1] Ping recibido desde coordinador
2026-10-20 14:25:59,736 [nodo1] Ping recibido desde coordinador
2026-10-20 14:26:04,751 [nodo1] Ping recibido desde coordinador
```

Figura 2. Pings del coordinador recibidos por nodo1

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python nodo.py --id nodo2 --puerto 50052
2026-10-20 14:26:24,804 [nodo2] Ping recibido desde coordinador
2026-10-20 14:26:29,814 [nodo2] Ping recibido desde coordinador
2026-10-20 14:26:34,826 [nodo2] Ping recibido desde coordinador
2026-10-20 14:26:39,840 [nodo2] Ping recibido desde coordinador
2026-10-20 14:26:44,857 [nodo2] Ping recibido desde coordinador
2026-10-20 14:26:49,869 [nodo2] Ping recibido desde coordinador
2026-10-20 14:26:54,879 [nodo2] Ping recibido desde coordinador
2026-10-20 14:26:59,894 [nodo2] Ping recibido desde coordinador
2026-10-20 14:27:04,906 [nodo2] Ping recibido desde coordinador
```

Figura 3. Pings del coordinador recibidos por nodo2

E3. Comunicación directa con un nodo

Qué se hizo: Se llamó al método Ping de nodo1, primero con una instrucción de Python y luego con el comando `python cliente.py ping localhost:50051`.

Qué se observa: En ambos casos nodo1 responde con su identificador (`nodo_id: nodo1`). Es la primera comprobación de que la llamada a procedimiento remoto funciona de extremo a extremo.

```
PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> (Set-ExecutionPolicy -Scope Process -ExecutionPol
icy RemoteSigned) ; (& c:\Users\ninod\proyectos\sd-almacenamiento-grpc\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python -c "import grpc, almacenamiento_pb
2 as pb, almacenamiento_pb2_grpc as g; s=g.NodoAlmacenamientoStub(grpc.insecure_channel('localhost:50
051')); print(s.Ping(pb.SolicitudPing(origen='prueba'), timeout=5))"
nodo_id: "nodo1"

(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc>
```

Figura 4. Llamada directa al stub gRPC de nodo1

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python cliente.py ping localhost:50051
Respuesta de nodo1: 0 archivo(s) guardado(s)
```

Figura 5. Ping a nodo1 desde el cliente de consola

E4. Subida con réplicas, listado y descarga

Qué se hizo: Se subieron ejemplo.txt y archivo_1mb.bin, se listaron sus réplicas y se descargó el archivo de 1 MB.

Qué se observa: ejemplo.txt (88 bytes) se guardó en nodo1 y nodo2. archivo_1mb.bin (1 048 576 bytes) viajó en 16 fragmentos de 64 KB a nodo3 (9,4 ms) y nodo1 (12,4 ms). Todas las subidas reportan "hash OK". La descarga desde nodo3 tardó 9,9 ms y devolvió el SHA-256 dfa44ac65c9d….

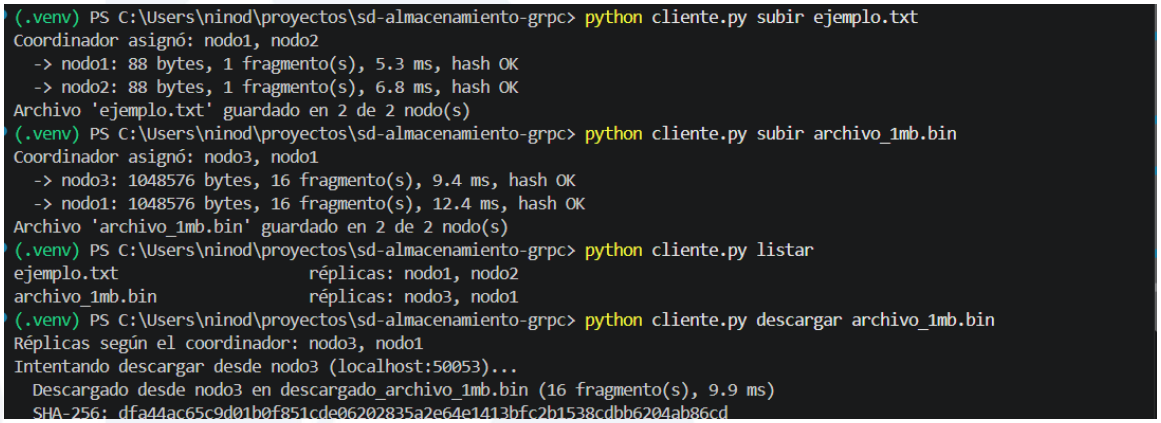


Figura 6. Subida a dos réplicas, listado y descarga por streaming

E5. Consulta de un archivo inexistente

Qué se hizo: Se ejecutó `python cliente.py info no_existe.pdf`.

Qué se observa: El coordinador responde con el código gRPC `NOT_FOUND` y el mensaje "El coordinador no conoce el archivo 'no_existe.pdf'". El cliente muestra el error de forma controlada y termina sin excepciones.

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python cliente.py info no_existe.pdf
ERROR gRPC [NOT_FOUND]: El coordinador no conoce el archivo 'no_existe.pdf'
```

Figura 7. Error controlado NOT_FOUND

E6. Caída de un nodo durante la descarga

Qué se hizo: ejemplo.txt estaba replicado en nodo1 y nodo2. Se detuvo nodo1 con Ctrl+C e inmediatamente se descargó el archivo con el destino copia_falla.txt.

Qué se observa: El cliente intenta nodo1, recibe `UNAVAILABLE` ("Falló nodo1 → se prueba la siguiente réplica") y descarga el archivo desde nodo2 en 7,1 ms. El contenido de copia_falla.txt es idéntico al de ejemplo.txt.

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc>    python cliente.py descargar ejemplo.txt --destino copia_
falla.txt
Réplicas según el coordinador: nodo1, nodo2
Intentando descargar desde nodo1 (localhost:50051)...
  Falló nodo1: [UNAVAILABLE] -> se prueba la siguiente réplica
Intentando descargar desde nodo2 (localhost:50052)...
  Descargado desde nodo2 en copia_falla.txt (1 fragmento(s), 7.1 ms)
  SHA-256: 66885ca86b57605bd26356b68a138f6d95c624a2db472efabca7b3a05d844002
```

Figura 8. Conmutación automática a la réplica tras la caída de nodo1

```
ejemplo.txt
1   HOLA COMO ESTAS ESTA ES UNA PRUEBA DE MI PROYECTO FINALLLLL DE MI 7MO SEMESTREEE JEJEJE

copia_falla.txt
1   HOLA COMO ESTAS ESTA ES UNA PRUEBA DE MI PROYECTO FINALLLLL DE MI 7MO SEMESTREEE JEJEJE
```

Figura 9. Contenido del archivo original (arriba) y de la copia descargada (abajo)

E7. Detección de la caída por el coordinador

Qué se hizo: Con el coordinador activo se detuvo nodo2.

Qué se observa: A las 14:28:02 el coordinador registra "Nodo nodo2 NO RESPONDE (caído)". Desde ese momento lo pone al final de la lista de réplicas para que los clientes usen primero los nodos vivos.

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python coordinador.py
2026-10-20 14:24:29,291 [coordinador] Coordinador escuchando en el puerto 50050
2026-10-20 14:24:41,110 [coordinador] Nodo registrado: nodo1 en localhost:50051
2026-10-20 14:24:51,278 [coordinador] Nodo registrado: nodo2 en localhost:50052
2026-10-20 14:28:02,100 [coordinador] Nodo nodo2 NO RESPONDE (caído)
```

Figura 10. El coordinador detecta la caída de nodo2

E8. Recuperación de los nodos

Qué se hizo: Se reinició el coordinador y luego los nodos.

Qué se observa: El coordinador conserva los nodos en su índice persistente. Cuando nodo1 (14:36:31) y nodo2 (14:36:36) se registran de nuevo, los reconoce y registra "VOLVIÓ a estar disponible".

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python coordinador.py
2026-10-20 14:36:17,239 [coordinador] Coordinador escuchando en el puerto 50050
2026-10-20 14:36:31,981 [coordinador] Nodo nodo1 VOLVIÓ a estar disponible (se registró de nuevo)
2026-10-20 14:36:36,253 [coordinador] Nodo nodo2 VOLVIÓ a estar disponible (se registró de nuevo)
```

Figura 11. El coordinador registra el regreso de los nodos

2. PRUEBAS DESDE LA INTERFAZ WEB

El panel web se comunica por HTTP con el gateway (FastAPI), y el gateway traduce cada petición a llamadas gRPC al coordinador y a los nodos.

E9. Panel: nodos, subida, información y descarga

Qué se hizo: Con los tres nodos activos se subieron desde el panel una fotografía y un documento de Word, se consultó la información de la foto y se descargó.

Qué se observa: Las tres tarjetas aparecen en verde. El documento .docx viajó en 23 fragmentos a nodo3 y nodo1 con "hash OK". La consulta Info muestra la foto en nodo1 y nodo2 con los mismos 970 252 bytes y el mismo SHA-256 (8bffc199b1f1...), y la descarga sale desde nodo1. Los nombres largos con espacios y ñ se manejan sin problema.

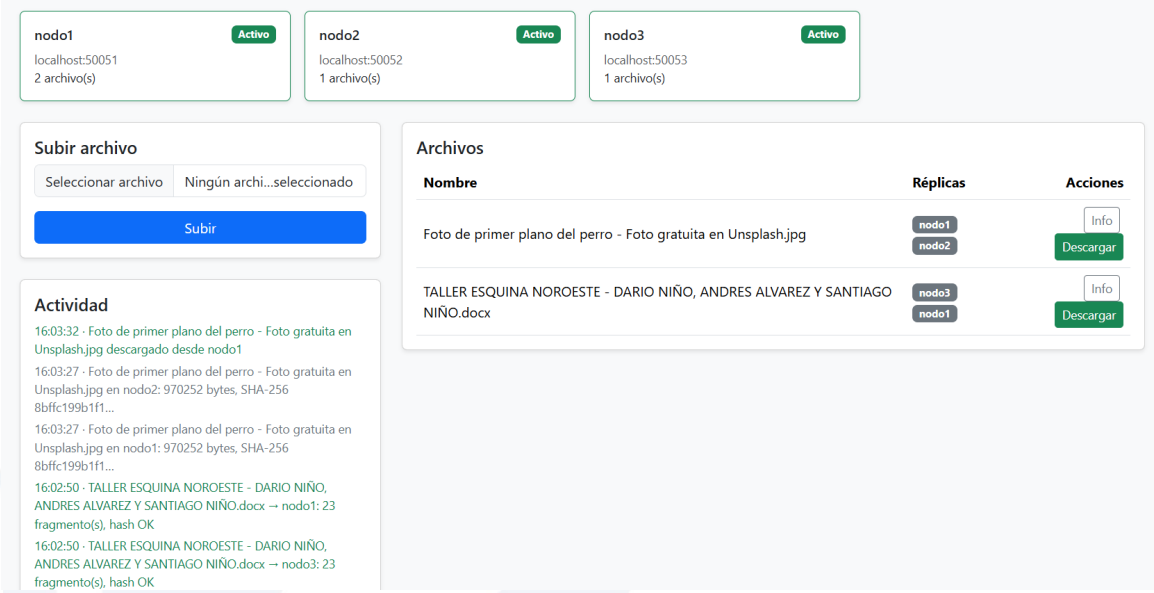


Figura 12. Panel web con los tres nodos activos y dos archivos replicados

E10. Nodo caído en el panel

Qué se hizo: Se detuvo nodo1 sin tocar la página.

Qué se observa: A las 16:06:25 la tarjeta de nodo1 pasa a rojo con la etiqueta "Caído", y su insignia aparece en rojo en los dos archivos que tenía. Ambos archivos siguen disponibles porque cada uno conserva otra réplica activa: la foto en nodo2 y el documento en nodo3.

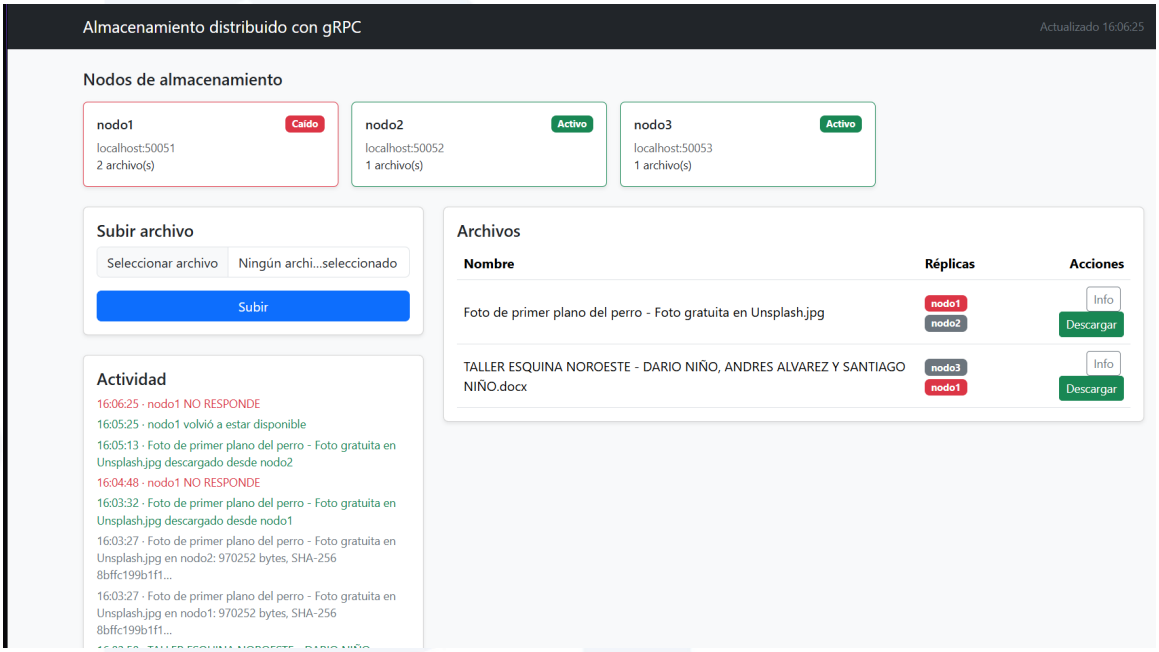


Figura 13. Panel con nodo1 caído y sus réplicas marcadas en rojo

E11. Historial de caída, descarga y recuperación

Qué se hizo: Se detuvo nodo1, se descargó la foto y después se volvió a iniciar nodo1.

Qué se observa: El historial de actividad registra en orden: 16:04:48 "nodo1 NO RESPONDE", 16:05:13 la foto descargada desde nodo2 (el coordinador ya había reordenado las réplicas) y 16:05:25 "nodo1 volvió a estar disponible".

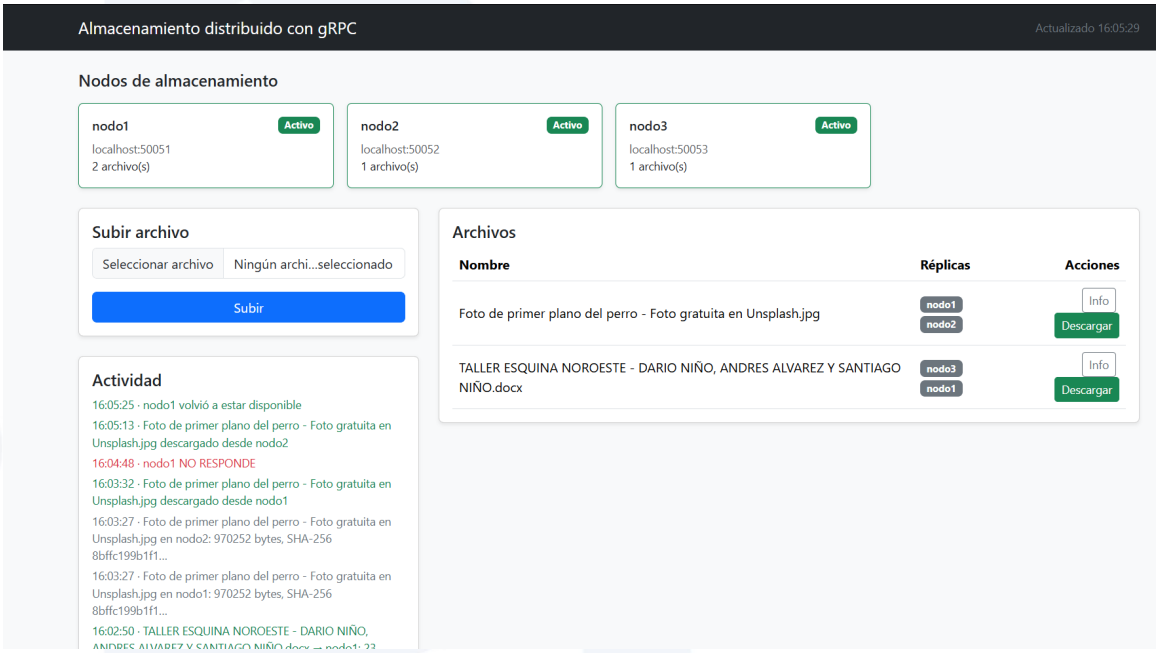


Figura 14. Historial de actividad con la caída y la recuperación de nodo1

E12. API: consulta exitosa

Qué se hizo: En la documentación automática (/docs) se ejecutó `GET /api/info/{nombre}` con el nombre de la foto.

Qué se observa: La API responde con código 200 y las dos réplicas, nodo1 y nodo2, con 970 252 bytes y el mismo SHA-256 completo (8bffc199b1f17fb13d86…f8754). Las dos copias son idénticas byte a byte.

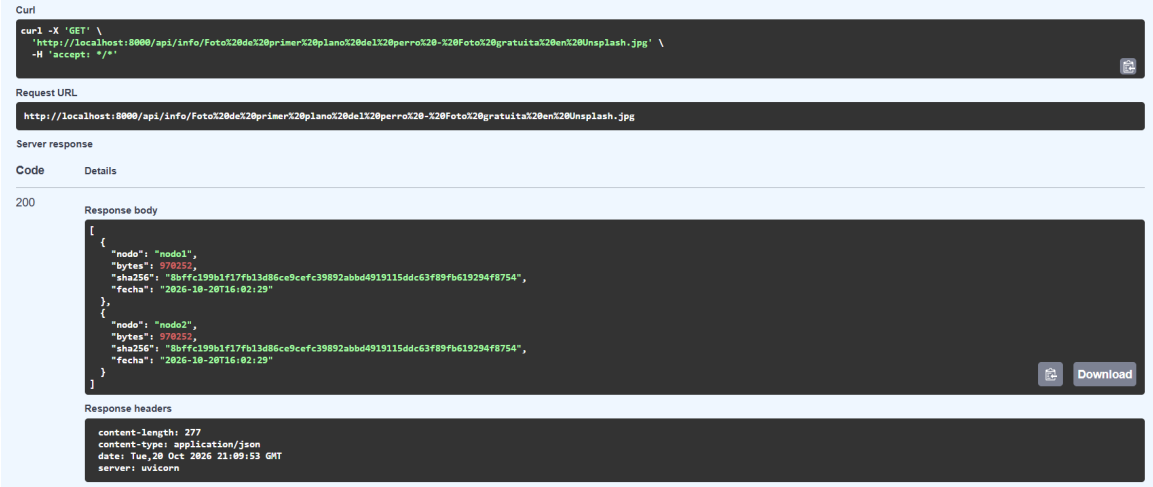


Figura 15. Consulta exitosa desde la documentación de la API

E13. API: archivo inexistente

Qué se hizo: Se ejecutó `GET /api/info/no_existe.pdf` desde /docs.

Qué se observa: La API responde con código 404 y el detalle "[NOT_FOUND] El coordinador no conoce el archivo 'no_existe.pdf'". El error se origina en gRPC y el gateway lo traduce a HTTP sin perder el mensaje.

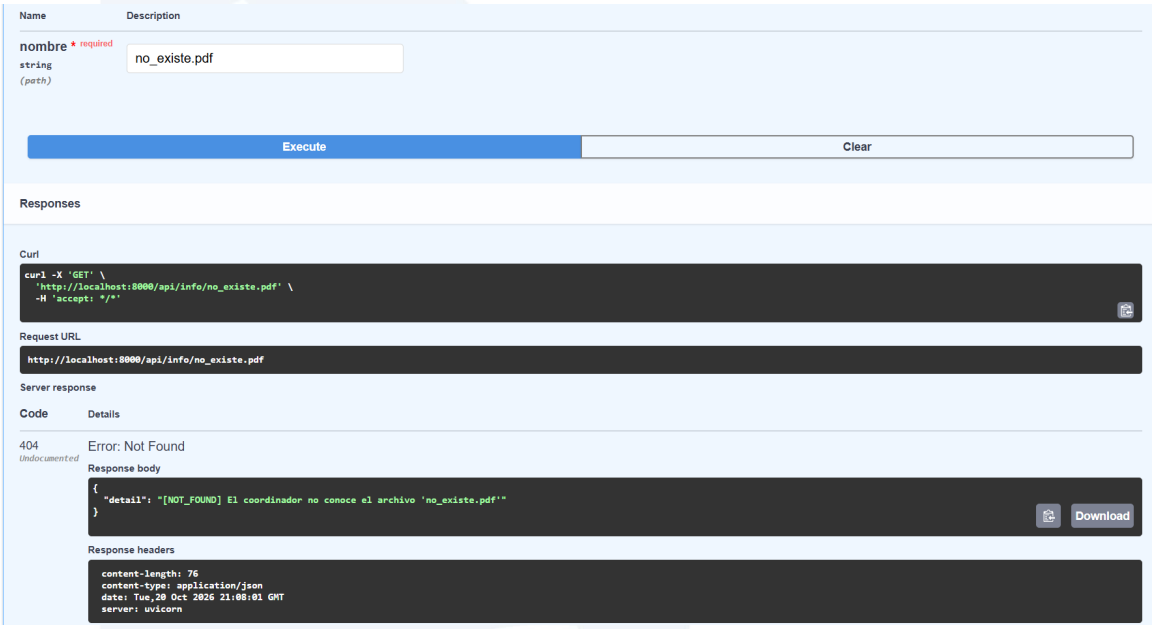


Figura 16. Error 404 traducido desde el código gRPC NOT_FOUND

3. MEJORAS IMPLEMENTADAS

Esta sección registra las mejoras incorporadas después del prototipo inicial, con la evidencia de su funcionamiento.

E14. Confirmación de subida

Qué se hizo: Antes, el coordinador registraba las réplicas al asignarlas, aunque la subida fallara. Se agregó el método `ConfirmarSubida`: la subida queda pendiente y el coordinador solo registra los nodos que el cliente confirma. Para probarlo se detuvo nodo3 y, antes de que el coordinador detectara la caída, se subió un archivo desde el panel.

Qué se observa: En el caso normal, la subida de una imagen queda "pendiente de confirmar" a las 16:35:20 y se confirma en nodo2 y nodo3 con sus 18 317 bytes. En el caso con falla, a las 16:42:28 la subida a nodo3 da `UNAVAILABLE`, la de nodo2 termina con "hash OK" y el coordinador confirma el archivo solo en nodo2. Dos segundos después detecta la caída de nodo3. La tabla muestra el archivo con una sola réplica y nodo3 no suma el archivo que no recibió.

```
2026-10-20 16:34:53,941 [coordinador] Nodo nodo1 VOLVIÓ a estar disponible (se registró de nuevo)
2026-10-20 16:34:57,687 [coordinador] Nodo nodo2 VOLVIÓ a estar disponible (se registró de nuevo)
2026-10-20 16:35:01,318 [coordinador] Nodo nodo3 VOLVIÓ a estar disponible (se registró de nuevo)
2026-10-20 16:35:20,176 [coordinador] Subida de 'imagen_2026-10-20_163518610.png' asignada a ['nodo2', 'nodo3'] (pendiente de confirmar)
2026-10-20 16:35:20,201 [coordinador] Subida de 'imagen_2026-10-20_163518610.png' confirmada en ['nodo2', 'nodo3'] (18317 bytes)
```

Figura 17. Registro del coordinador: subida pendiente y luego confirmada

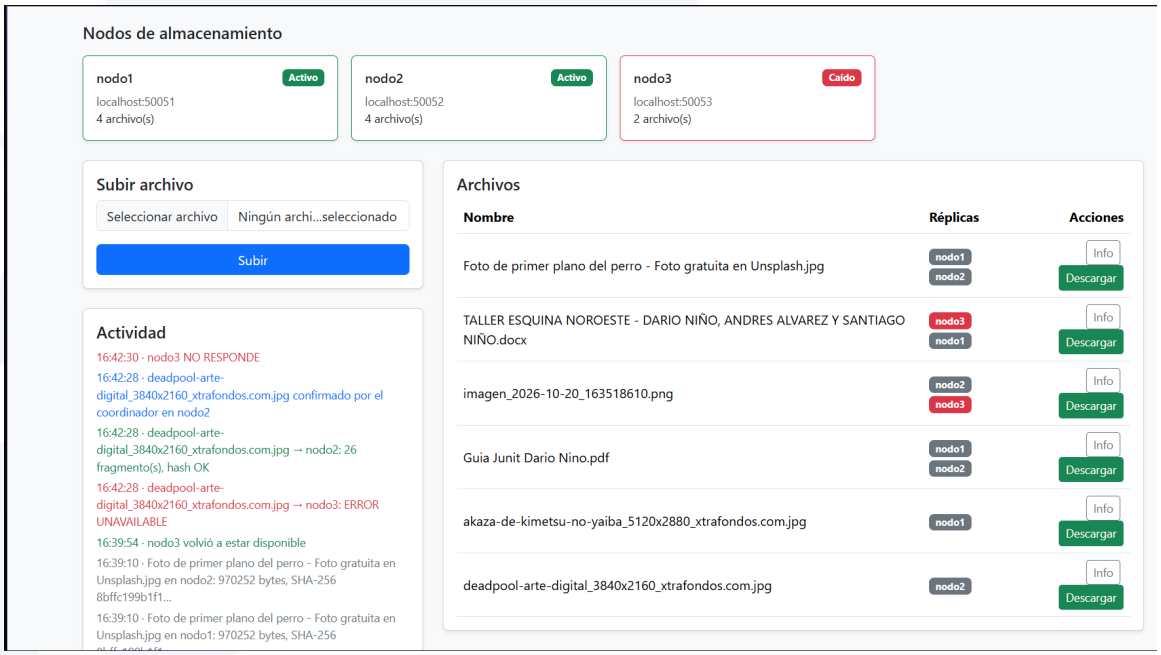


Figura 18. Subida con un nodo caído: solo se registra la réplica exitosa

E15. Eliminación de archivos

Qué se hizo: Se agregó el método `EliminarArchivo` en los nodos y en el coordinador, el endpoint `DELETE /api/archivos/{nombre}` y un botón Eliminar en el panel con confirmación. El coordinador ordena a cada nodo con réplica que borre su copia y luego quita el archivo del índice. Se eliminaron archivos con todos los nodos activos y después con nodo2 detenido.

Qué se observa: A las 18:59:29 la foto se elimina de sus dos réplicas, nodo1 y nodo2, y a las 18:59:55 se elimina un archivo que solo tenía réplica en nodo1. A las 19:00:26 nodo2 deja de responder; a las 19:00:32 el PDF de la guía se elimina de nodo1 y el panel advierte que no se pudo borrar en nodo2 por estar caído. Esa copia huérfana la limpia después la resincronización (E21). A las 19:00:45 nodo2 vuelve a estar disponible.

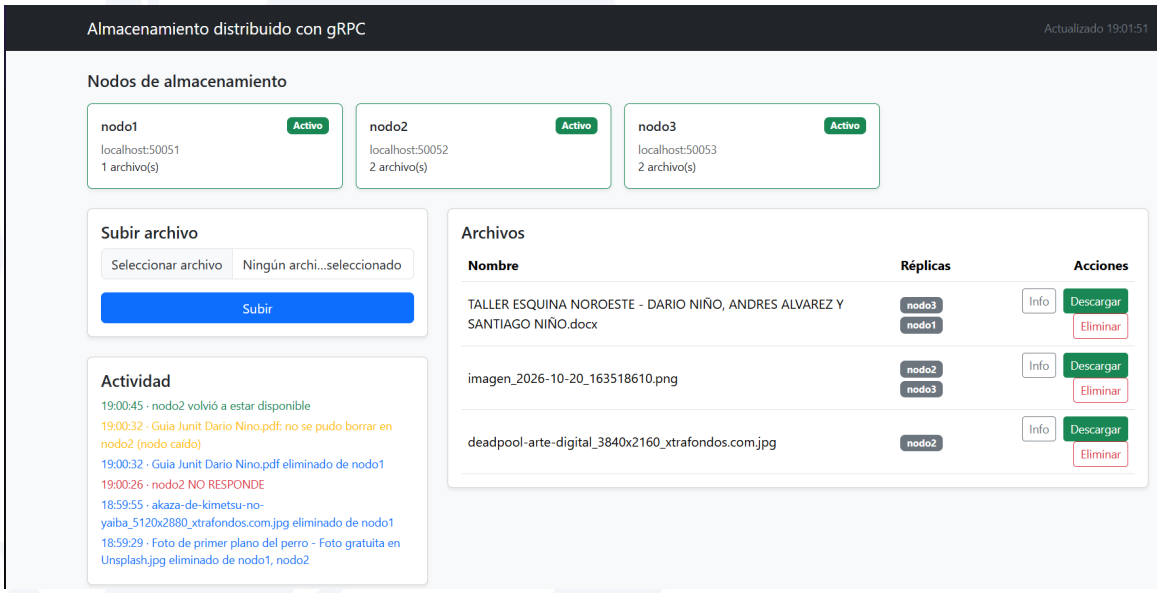


Figura 19. Eliminación de archivos con todos los nodos activos y con nodo2 caído

4. DESPLIEGUE Y ACCESO REMOTO

Para que el sistema funcione entre equipos de redes distintas se usa Tailscale, una red privada virtual que asigna a cada dispositivo una dirección fija del rango 100.64.0.0/10 y cifra el tráfico con WireGuard. El PC central quedó con la dirección 100.123.23.123. Los nodos se anuncian ante el coordinador con esa dirección y el gateway escucha en todas las interfaces para atender a otros dispositivos de la red.

E16. Acceso remoto al panel por Tailscale

Qué se hizo: Con el coordinador, tres nodos y el gateway en el PC central, se abrió el panel desde un celular conectado a la misma red de Tailscale, en la dirección http://100.123.23.123:8000.

Qué se observa: El registro del gateway muestra peticiones desde dos direcciones de Tailscale: 100.123.23.123 (el propio PC) y 100.87.69.47 (el celular). Desde el celular se cargaron la página y las consultas `/api/nodos` y `/api/archivos`, todas con código 200. Los 404 corresponden a los íconos que Safari pide automáticamente, que el proyecto no tenía en ese momento.

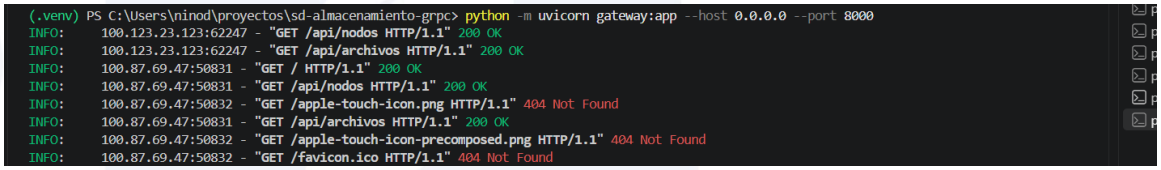


Figura 20. Peticiones del celular al gateway a través de Tailscale

E17. Panel en una red lenta

Qué se hizo: Se usó el panel desde el celular con datos móviles (4G) mientras el PC central también salía a internet por el punto de acceso de ese mismo celular. En esa red Tailscale no logra una conexión directa y el tráfico pasa por un servidor de relevo.

Qué se observa: Varias consultas se cortaron antes de completarse y Safari reportó "Load failed". La versión del panel de ese momento borraba los datos ante el primer fallo y mostraba "Coordinador no disponible", aunque el sistema seguía funcionando. A partir de esta prueba se mejoró el panel: cada consulta tiene un plazo máximo de 10 segundos, no se envía una nueva mientras la anterior no termina, se conservan los últimos datos mostrando "Reconectando…" y el error solo aparece tras tres fallos seguidos.

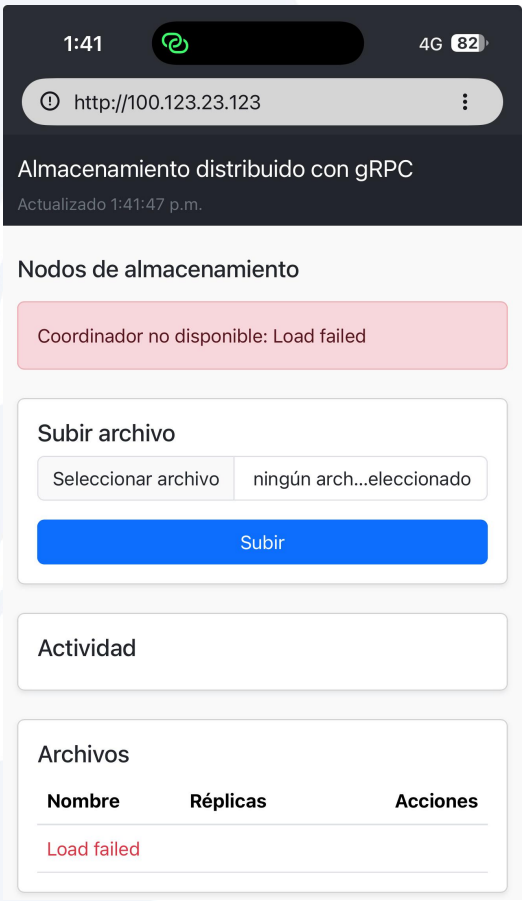


Figura 21. Panel en el celular con consultas cortadas por la red lenta (versión anterior a la mejora)

E18. Nodo en un segundo computador

Qué se hizo: Se ejecutó el sistema en dos computadores conectados por Tailscale. El PC central (100.123.23.123) corrió el coordinador, el gateway, nodo1 y nodo3 con `python iniciar.py --ids nodo1,nodo3`. Un segundo computador corrió solo nodo2 con `python iniciar.py --rol nodo --id nodo2`, con su IP de Tailscale y la del coordinador definidas en su archivo config_local.py.

Qué se observa: A las 22:57:21 el coordinador registra a nodo2 conectado en la dirección 100.114.167.30:50052, que pertenece al segundo computador: la llamada gRPC RegistrarNodo viajó de un equipo al otro. Trescientos milisegundos después, el primer ping del coordinador hacia nodo2 no obtuvo respuesta y lo marcó como caído; el siguiente ping, cinco segundos más tarde, sí respondió y el coordinador registró que volvió a estar disponible. La conexión de entrada hacia el segundo equipo tardó unos segundos en quedar habilitada, y el chequeo de salud lo detectó y se recuperó sin intervención.

```
(.venv) PS C:\Users\ninod\proyectos\sd-almacenamiento-grpc> python iniciar.py --ids nodo1,nodo3
gateway     | INFO:     127.0.0.1:54547 - "GET / HTTP/1.1" 200 OK
gateway     | INFO:     127.0.0.1:54547 - "GET /api/sesion HTTP/1.1" 200 OK
gateway     | INFO:     127.0.0.1:54547 - "GET /favicon.ico HTTP/1.1" 204 No Content
coordinador | 2026-10-25 22:57:21,457 [coordinador] Nodo nodo2 conectado en 100.114.167.30:50052
coordinador | 2026-10-25 22:57:21,764 [coordinador] Nodo nodo2 NO RESPONDE (caído)
coordinador | 2026-10-25 22:57:26,902 [coordinador] Nodo nodo2 VOLVIÓ a estar disponible
```

Figura 22. El coordinador registra y vigila a nodo2, que corre en otro computador

5. TOLERANCIA A FALLOS: RE-REPLICACIÓN Y RESINCRONIZACIÓN

Cuando un nodo cae, sus archivos quedan con menos copias vivas de las debidas (sub-replicados). El coordinador espera un periodo de gracia de 30 segundos por si el nodo vuelve; si sigue caído, ordena a un nodo con copia viva enviarla a otro nodo sano con el método `ReplicarHacia`. Los datos viajan de nodo a nodo y el coordinador solo registra la réplica nueva después de comparar su SHA-256 con el de la subida original.

Cuando el nodo caído vuelve, el coordinador le pide su `Inventario` (los archivos que tiene de verdad en disco, con el hash recalculado), lo compara con el índice y corrige las diferencias: borra las copias huérfanas, descarta las dañadas o perdidas y, si un archivo quedó con más copias de las debidas, elimina la sobrante del nodo más cargado.

E19. Re-replicación automática

Qué se hizo: Con los tres nodos activos y un archivo guardado en NODO1 y NODO2, se detuvo NODO2. Después se detuvo también NODO1, se descargó el archivo y finalmente se volvieron a iniciar los dos nodos.

Qué se observa: La actividad del sistema registra la secuencia completa. A las 01:12:38 NODO2 deja de responder y a las 01:13:09, 31 segundos después, el archivo se re-replica de NODO1 a NODO3 con el hash verificado: el sistema esperó el periodo de gracia antes de actuar. A las 01:14:02 cae también NODO1 y, aun así, a las 01:14:06 el usuario descarga el archivo: la única copia viva es la que creó la re-replicación en NODO3. A las 01:14:34 el coordinador avisa que el archivo tiene una sola copia viva y no hay otro nodo disponible para copiarlo. Luego NODO1 y NODO2 vuelven y el sistema sigue recibiendo archivos con normalidad.

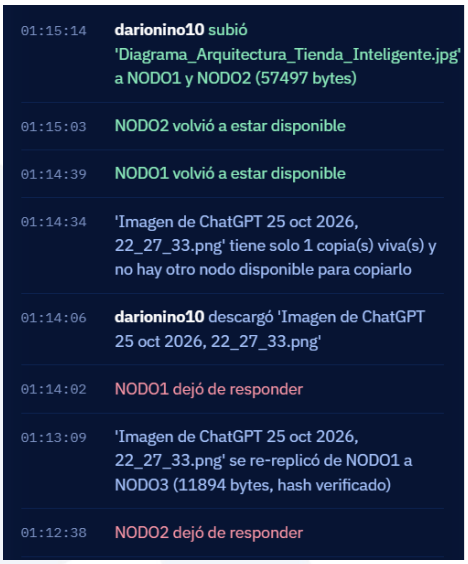


Figura 23. Secuencia de caídas, re-replicación, descarga y aviso de riesgo

E20. Estado después de la re-replicación

Qué se hizo: Con los tres nodos de nuevo activos, se revisó el panel antes de incorporar la resincronización.

Qué se observa: El archivo re-replicado aparece con tres réplicas, en NODO1, NODO2 y NODO3, todas en verde: la copia creada durante la caída se conserva y el nodo que volvió todavía tiene la suya. Esta sobre-replicación es la que resuelve la resincronización (E21).



Figura 24. Panel con el archivo re-replicado en tres nodos

E21. Resincronización al volver un nodo

Qué se hizo: Con la resincronización implementada, se detuvo NODO1, que tenía copia de tres archivos, y se esperó la re-replicación. Después se volvió a iniciar NODO1.

Qué se observa: A las 12:26:06 NODO1 deja de responder. Entre las 12:26:37 y las 12:26:38, unos 31 segundos después, los tres archivos se re-replican con el hash verificado: dos de NODO2 a NODO3 y uno de NODO3 a NODO2. A las 12:27:26 NODO1 vuelve a estar disponible; el coordinador revisa su inventario y, dos segundos después, encuentra los tres archivos con tres copias vivas. En cada uno elimina la copia sobrante del nodo más cargado (NODO3, NODO2 y NODO1), y todos vuelven a tener exactamente dos réplicas, sin intervención manual.



Figura 25. Caída, re-replicación, regreso del nodo y eliminación de copias sobrantes

E22. Indicador de réplicas en riesgo

Qué se hizo: Se detuvo NODO1 y se observó la tabla de archivos del panel durante el periodo de gracia, antes de que actuara la re-replicación.

Qué se observa: La tarjeta de NODO1 aparece en rojo como "Inactivo" y el encabezado de la tabla indica "4 en total · 2 en riesgo". Los dos archivos que tenían copia en NODO1 muestran la etiqueta amarilla "En riesgo · 1 de 2 copias" y un borde amarillo, mientras que el archivo guardado en NODO2 y NODO3 se ve normal. Al pasar el puntero sobre la etiqueta, el panel explica que el sistema creará una copia nueva si el nodo no vuelve en 30 segundos.

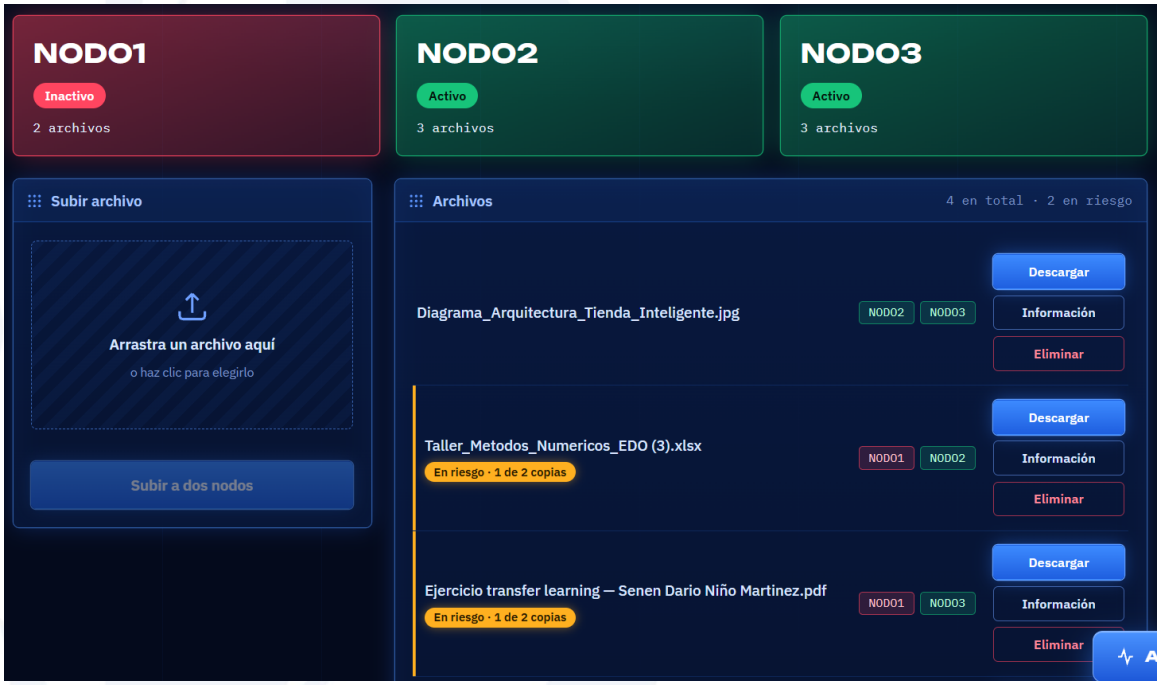


Figura 26. Archivos marcados en riesgo mientras NODO1 está inactivo

6. CUENTAS, PERMISOS Y ACCESO MÓVIL

El panel exige iniciar sesión. Las contraseñas se guardan con hash scrypt y una sal por usuario, la sesión viaja en una cookie HttpOnly y el gateway responde 401 a cualquier petición sin sesión válida. La primera cuenta del sistema es la del administrador, y solo ese rol puede eliminar archivos: el gateway responde 403 a cualquier otro usuario que lo intente.

E23. Acceso móvil con un usuario sin permisos de eliminación

Qué se hizo: Se abrió el panel desde un celular conectado por Wi-Fi a la red de Tailscale, en la dirección http://100.123.23.123:8000, con la sesión de un usuario sin rol de administrador.

Qué se observa: El panel carga y se actualiza con normalidad (12:32:35), con los tres nodos activos y los cuatro archivos. A diferencia de la prueba con datos móviles (E17), con Wi-Fi la conexión es estable. Cada archivo muestra solo los botones Descargar e Información: el botón Eliminar no aparece, porque esa acción está reservada para el administrador.

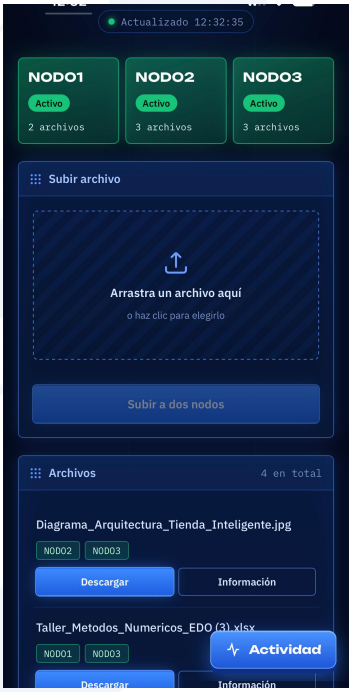


Figura 27. Panel en el celular con un usuario sin permiso para eliminar